

Artificial Intelligence Midterm Exam

Ultimate Solution & Concept Guide

Answer Key & Explanations

PART-1 (20 Marks)

1. For a graph search problem, if the cost of every action (move) is the same, breadth first search is guaranteed to return an optimal solution.
(A) True (B) False

Correct Answer & Simple Explanation

Correct Option: (A) True

Why? Imagine pouring water on the start node of a graph. The water spreads outward evenly, layer by layer (which is exactly what BFS does). The very first time the water touches the goal, it *must* have taken the shortest possible route. If all steps cost the same, the path with the fewest steps is always the optimal one!

2. For a 3×3 tic-tac-toe game, there are _____ different paths (from start to the end of game) in its state space graph.
(A) 3^9 (B) 9^3 (C) $8!$ (D) None of the given options

Correct Answer & Simple Explanation

Correct Option: (D) None of the given options

Why? Let's count! The first player has 9 empty boxes to choose from. The second player has 8 empty boxes left. The next has 7, and so on. Mathematically, this is $9 \times 8 \times 7 \times \dots \times 1$, which is $9!$ (362,880). Since $9!$ is not an option, D is correct.

3. If symmetry of initial symbol placement is considered, the above tic-tac-toe game has _____ paths through its state space graph.
(A) $3 \times 7!$ (B) $12 \times 7!$ (C) $8 \times 7!$ (D) None of the given options

Correct Answer & Simple Explanation

Correct Option: (D) None of the given options

Why? Because a tic-tac-toe board is a symmetrical square, opening in the top-left corner is strategically the exact same as opening in the bottom-right corner. There are actually only **3** unique opening moves: a corner, an edge-middle, or the dead center. After that 1st move, the remaining 8 boxes are filled normally ($8!$). Total unique paths = $3 \times 8!$, which can also be written as $24 \times 7!$. This is not listed in the options.

4. Which of the following is true about data-driven search?
(A) It is also called backward chaining
(B) It starts at the goal node(s)
(C) It involves searching from facts to the goal(s)
(D) It is feasible when goal(s) of a problem are known.

Correct Answer & Simple Explanation

Correct Option: (C) It involves searching from facts to the goal(s)

Why? Think of a detective. If they start with clues (data/facts) and follow them to find out who the killer is (the goal), that is a *data-driven* search (Forward chaining). If they start with a suspect (the goal) and look backward for clues to prove it, that is *goal-driven* (Backward chaining).

5. For a shallower search tree, depth first iterative deepening search compared with depth first search, in general, is:

(A) More efficient (B) Equally well (C) Optimal (D) Inefficient

Correct Answer & Simple Explanation

Correct Option: (D) Inefficient

Why? Iterative Deepening works by restarting the search over and over (Search to depth 1, restart. Search to depth 2, restart...). This is great for infinitely deep trees so you don't get lost. But if the tree is already shallow, standard DFS would just finish it quickly. Restarting over and over on a tiny tree is just wasted, inefficient work.

6. For the 8-puzzle problem, the state space has _____ states accessible from any random start state.

(A) $8!$ (B) $9!$ (C) $9!/2$ (D) None of the given options

Correct Answer & Simple Explanation

Correct Option: (C) $9!/2$

Why? Imagine taking an 8-puzzle, popping all the tiles out with a screwdriver, and snapping them back in randomly. Half of those random combinations are mathematically impossible to solve by sliding! The state space is split into two disconnected universes. You can only ever reach exactly half ($9!/2$) of the total combinations without physically breaking the puzzle.

7. $\alpha\beta$ search is always better than the minimax search.

(A) True (B) False

Correct Answer & Simple Explanation

Correct Option: (B) False

Why? Alpha-Beta is just Minimax *with a shortcut*. But shortcuts only work if you find them! If you have terrible luck and examine the worst possible moves first, the shortcut conditions never trigger. In that worst-case scenario, Alpha-Beta does the exact same amount of work as regular Minimax. So it's not *always* better.

8. α cut-off means:

(A) Prune further possibilities below the current MAX node
(B) Prune further possibilities below the current MIN node

Correct Answer & Simple Explanation

Correct Option: (B) Prune further possibilities below the current MIN node

Why? Alpha (α) is the best score the MAX player knows they can absolutely guarantee so far. If the MIN player is exploring a branch and finds a move that results in a score

worse than α , MIN knows MAX will never let the game reach this point anyway. So, MIN stops searching and prunes the rest of its branch.

9. If d represents depth and b , the average branching factor, the worst-case space complexity of $\alpha\beta$ search can be characterized as:

(A) $O(b^d)$ (B) $O(b \log d)$ (C) $O(d^b)$ (D) $O(bd)$

Correct Answer & Simple Explanation

Correct Option: (D) $O(bd)$

Why? Alpha-Beta search dives straight down to the bottom of the tree (Depth-First). It doesn't need to remember the entire tree, only the single vertical path it is currently walking down (depth d) and the siblings at each step (branching b). Multiplying them gives $O(bd)$, making it very memory efficient!

10. Admissibility of a heuristic always implies:

(A) Informedness (B) Monotonicity (C) Irrevocability (D) None of the given options

Correct Answer & Simple Explanation

Correct Option: (D) None of the given options

Why? Admissibility just means "never overestimate the cost". Monotonicity (consistency) is a much stricter rule that says "never overestimate AND step costs must logically align". Therefore, Monotonicity implies Admissibility, but Admissibility does *not* imply Monotonicity. (Like how running a marathon implies you sweat, but sweating doesn't imply you ran a marathon).

11. In best first search, which of the following is true:

(A) A set of sorted paths is added to the queue
(B) Entire queue is sorted each time a new path is added
(C) Chronological backtracking is done to switch to promising paths
(D) All the nodes with a specified heuristic value(s) are explored first

Correct Answer & Simple Explanation

Correct Option: (B) Entire queue is sorted each time a new path is added

Why? Best First Search uses a Priority Queue. Imagine a hospital waiting room. Every time a new patient (node) walks in, the nurse re-evaluates everyone's urgency (heuristic) and re-sorts the entire line so the absolute most critical patient is seen next.

12. _____ is a conceptual representation, while _____ is a concrete one.

(A) Search Tree, State Space (B) State Space, Search Tree

Correct Answer & Simple Explanation

Correct Option: (B) State Space, Search Tree

Why? The "State Space" is the abstract mathematical map of every possible thing that *could* happen. The "Search Tree" is the concrete data structure your computer actually builds in its RAM as it explores paths step-by-step.

13. Consider a search tree with depth $d = 3$ and branching factor $b = 2$ the number of nodes explored by depth first iterative deepening search in worst case would be:

(A) 8 (B) 15 (C) 26 (D) 22

Correct Answer & Simple Explanation

Correct Option: (C) 26

Why? DFIDS recounts the top layers over and over.

Iteration 0 (Depth 0): Explores 1 node.

Iteration 1 (Depth 1): Explores $1 + 2 = 3$ nodes.

Iteration 2 (Depth 2): Explores $1 + 2 + 4 = 7$ nodes.

Iteration 3 (Depth 3): Explores $1 + 2 + 4 + 8 = 15$ nodes.

Total nodes generated: $1 + 3 + 7 + 15 = 26$.

14. **For certain problems, e.g. fractional knapsack problem, hill climbing is always guaranteed to give optimal solution.**

(A) True (B) False

Correct Answer & Simple Explanation

Correct Option: (A) True

Why? Because you can cut items into fractions, you don't have to worry about awkward gaps in your bag. You simply take the item with the highest "Value-per-Kg", fill as much as you can, and move to the next. This local greedy choice (hill climbing) perfectly builds the global optimal solution.

15. **Heuristic evaluation function for the uniform cost search may be characterized as:**

(A) $f(n) = g(n) + h(n)$; where $h(n) = 0$

(B) $f(n) = g(n) + h(n)$; where both $g(n)$ and $h(n)$ are non-negative

(C) $f(n) = g(n) + h(n)$

(D) None of the given options

Correct Answer & Simple Explanation

Correct Option: (A) $f(n) = g(n) + h(n)$; where $h(n) = 0$

Why? Uniform Cost Search (UCS) only cares about the actual distance traveled so far ($g(n)$). It has absolutely zero "intuition" or "guesses" about how far the goal is. Because it lacks this intuition, its heuristic value $h(n)$ is always exactly 0.

16. **Admissibility is a stricter requirement than monotonicity.**

(A) True (B) False

Correct Answer & Simple Explanation

Correct Option: (B) False

Why? As established in Question 10, Monotonicity (Consistency) is the strict one! It requires the heuristic to follow the triangle inequality law at every single step, whereas Admissibility only requires that the final estimated cost to the goal isn't overpriced.

17. **TSP optimization problem is an NP hard problem.**

(A) True (B) False

Correct Answer & Simple Explanation

Correct Option: (A) True

Why? The Traveling Salesperson Problem asks for the shortest path connecting N

cities. To absolutely guarantee the shortest path, a computer has to check almost every possible permutation. For 50 cities, there are more routes than atoms in the universe. It is a classic NP-Hard problem!

18. **A larger tournament size would result in a more diverse offspring population.**
(A) True (B) False

Correct Answer & Simple Explanation

Correct Option: (B) False

Why? Imagine a talent show. If you pick 2 random people from the crowd and choose the best singer, you get a decent variety of okay singers (High Diversity). If you pick 100 random people and choose the best singer (Large Tournament), you will almost always pick a professional. The population will quickly fill up with only clones of the professionals, completely destroying diversity!

19. **Genetic algorithm can be characterized as a powerful variant of:**
(A) Uniform cost search (B) Breadth first search
(C) Hill climbing search (D) None of the given options

Correct Answer & Simple Explanation

Correct Option: (C) Hill climbing search

Why? Genetic Algorithms are basically "Parallel Stochastic Hill Climbing". Instead of one climber looking for the peak, you drop a whole population of climbers. They look around randomly (mutation) and share their coordinates with each other to jump to better peaks (crossover).

20. **Mutation helps ensure that _____ is present in the offspring population.**
(A) Stagnation (B) Diversity (C) Inversion (D) None of the given options

Correct Answer & Simple Explanation

Correct Option: (B) Diversity

Why? If nobody in a population has the gene for "blue eyes", no amount of crossover (mating) will ever produce blue eyes. The gene is lost. Mutation randomly flips genes, spontaneously creating new traits (like blue eyes) out of thin air, ensuring the gene pool stays fresh and diverse!

PART-2 (50 Marks)

Q.1. [1+2+4+3]

A farmer wants to move himself, a fox, a goose and some grain across a river.

Using the 4-tuple representation for a problem state as $\langle a_1, a_2, a_3, a_4 \rangle$, where the indices 1 through 4 represent the farmer, the fox, the goose, and the grain, respectively, and $a_i \in \{L, R\}$.

Detailed Solution

(a) Start and goal states:

Start state: $\langle L, L, L, L \rangle$

Goal state: $\langle R, R, R, R \rangle$

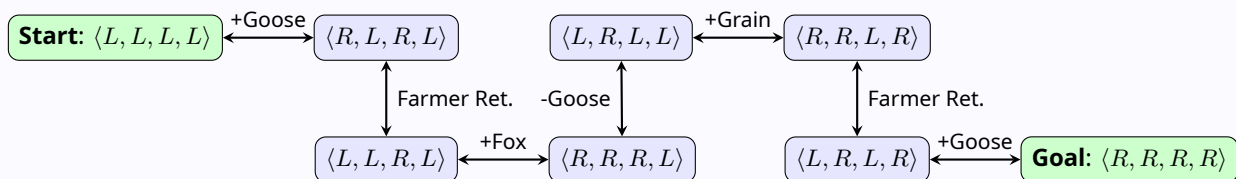
(b) States that must be avoided:

We must avoid states where the fox and goose are together without the farmer, or the goose and grain are together without the farmer.

1. $\langle R, L, L, L \rangle$ or $\langle R, L, L, R \rangle$ — (Farmer is Right, Fox & Goose are Left)
2. $\langle L, R, R, R \rangle$ or $\langle L, R, R, L \rangle$ — (Farmer is Left, Fox & Goose are Right)
3. $\langle R, R, L, L \rangle$ or $\langle R, L, L, L \rangle$ — (Farmer is Right, Goose & Grain are Left)
4. $\langle L, L, R, R \rangle$ or $\langle L, R, R, R \rangle$ — (Farmer is Left, Goose & Grain are Right)

(c) State Space Graph Diagram:

(Showing only permissible, non-fatal states and moves)



(d) Sequence of moves to reach the goal:

$\langle L, L, L, L \rangle \xrightarrow{+Goose} \langle R, L, R, L \rangle \xrightarrow{-Farmer} \langle L, L, R, L \rangle \xrightarrow{+Fox} \langle R, R, R, L \rangle \xrightarrow{-Goose} \langle L, R, L, L \rangle \xrightarrow{+Grain} \langle R, R, L, R \rangle \xrightarrow{-Farmer} \langle L, R, L, R \rangle \xrightarrow{+Goose} \langle R, R, R, R \rangle$

□ Concept Breakdown (Never Forget This!)

What is a "State Space Representation"?

Imagine a video game where you can pause at any given second. The exact location of every character, item, and door at that frozen second is a "State". The "State Space" is simply the map of every single possible paused screen in the game, and the arrows connect screens based on legal button presses (moves). Here, a 4-letter tuple maps exactly where our 4 characters are!

Q.2. [4+6]

Detailed Solution

(a) Differentiate between informedness and admissibility:

Admissibility: A heuristic $h(n)$ is admissible if it *never overestimates* the true cost to reach

the goal from node n . It is always optimistic. Mathematically, $h(n) \leq h^*(n)$.

Informedness: If we have two admissible heuristics, h_1 and h_2 , and $h_1(n) \geq h_2(n)$ for all nodes n , then h_1 is said to be *more informed* than h_2 . The closer a heuristic is to the true cost without going over, the more informed it is, and the fewer nodes it will waste time expanding.

(b) Show that monotonicity implies admissibility:

A heuristic is monotonic (consistent) if the estimated cost of reaching the goal from n is no greater than the step cost to a neighbor n' plus the estimated cost from n' : $h(n) \leq c(n, n') + h(n')$

To prove admissibility, we apply this iteratively down the actual shortest path to the goal node G . Let the path be: $n \rightarrow n_1 \rightarrow n_2 \cdots \rightarrow G$.

$$h(n) \leq c(n, n_1) + h(n_1)$$

$$h(n_1) \leq c(n_1, n_2) + h(n_2)$$

...

$$h(n_k) \leq c(n_k, G) + h(G)$$

Summing these up and canceling intermediate h terms (telescoping sum), and knowing that at the goal $h(G) = 0$, we get:

$$h(n) \leq \text{Total Path Cost to Goal}$$

$$h(n) \leq h^*(n), \text{ which is the exact definition of Admissibility!}$$

□ Concept Breakdown (Never Forget This!)

Heuristics: Admissibility vs. Monotonicity

Imagine you are driving to a city.

Admissible means your GPS predicts 50 miles, but the real winding road is 70 miles. It didn't overestimate, so it's admissible.

Monotonicity is stricter. It means your GPS must never contradict itself. If it says 50 miles left, and you drive 10 miles forward, it *must* now say 40 miles or less. If it suddenly says 45 miles, it means you took a step costing 10, but the distance only dropped by 5—breaking the triangle inequality. Consistent (monotonic) heuristics guarantee the GPS behaves logically!

Q.3. [2+4+4]

Consider the pseudo code for the best first search algorithm...

Detailed Solution

(a) Does the pseudo code work fine for finding the optimal path?

No.

(b) Change the pseudo code:

The issue is that it orders the 'open' list based purely on the heuristic merit $h(n)$ (like a Greedy search). To be optimal (like A*), it must order based on the total estimated path cost $f(n)$.

Correction: Replace the highlighted line re-order states on open by heuristic merit with:

re-order states on open by $f(n) = g(n) + h(n)$

(c) Modify the given pseudo code if the heuristic is consistent:

If a heuristic is consistent (monotonic), the first time A* pulls a node from the 'open' list to the 'closed' list, it is mathematically guaranteed to have found the absolute shortest path to that node. Therefore, we never have to check if a "shorter path" was found to a node already on 'closed'.

Correction: Remove this entire block from the pseudo code:

the child is already on closed:

begin

if the child was reached by a shorter path then

begin

remove the state from closed

add the child to open

end

end

□ Concept Breakdown (Never Forget This!)

The Magic Formula: $f(n) = g(n) + h(n)$

Why does A* use this?

$g(n)$ is the *past*: How much gas have I burned to get here?

$h(n)$ is the *future*: How much gas do I guess I need to reach the finish?

If you only sort by the future $h(n)$, you might take a terrible detour that just happens to point toward the goal right now. Adding the past $g(n)$ keeps the algorithm honest and ensures it finds the true shortest total trip!

Q.4.

Detailed Solution

(a) Alpha-Beta Pruning Performance:

Alpha-Beta pruning improves the Minimax algorithm by ignoring branches of the game tree that can't possibly affect the final decision. It maintains two bounds: α (the minimum score the MAX player is assured of) and β (the maximum score the MIN player is assured of). By pruning irrelevant branches, it heavily speeds up performance and memory.

Best Case Node Evaluation: $O(b^{d/2})$ (Happens when the best moves are generated first, allowing maximum pruning).

Worst Case Node Evaluation: $O(b^d)$ (Happens when nodes are ordered worst-to-best, meaning zero pruning occurs and it checks every node just like standard Minimax).

(b) Alpha-Beta Tree Trace and Final (α, β) Values:

Tracing depth-first, left-to-right updating α, β bounds.

At node **a** (MIN), it evaluates branch **b** which guarantees a 7. **a** sets its $\beta = 7$.

It then looks down branch **c**. Node **c** (MAX) evaluates **f** which returns 7. **c** sets its $\alpha = 7$.

Now, α at **c** is 7, and β passed down from **a** is 7. Because $\alpha \geq \beta$ ($7 \geq 7$), **c** realizes: "Wait, the MIN player above me will NEVER choose me if I try to push the score any higher than 7, because MIN already has a 7 guaranteed on the left."

Thus, pruning occurs! Node **g** and its children **n, o** are **pruned** and never evaluated.

Final move selection for MIN: b

Node	Final (α, β)
a	$(-\infty, 7)$
b	$(7, \infty)$
c	$(7, 7)$ (Prunes g)
d	$(-\infty, 1)$
e	$(1, 7)$
f	$(-\infty, 7)$
g	PRUNED
h	$(5, \infty)$
i	$(1, 5)$
j	$(9, \infty)$
k	$(7, 9)$
l	$(7, 7)$
m	$(8, 7)$
n	PRUNED
o	PRUNED

□ Concept Breakdown (Never Forget This!)

How Pruning Works in Real Life:

Imagine you are buying a car (You want the MINIMUM price).

Dealership B offers you a car for \$7,000 (This is $\beta = 7$).

You walk into Dealership C. The salesman says, "Let me check with my manager (MAX), but our prices start at \$7,000 ($\alpha = 7$), and we might charge more."
Do you wait for the manager to come back with a higher price? No! You walk out immediately (Pruning) because you already have a guaranteed \$7,000 deal across the street!

Q.5. [2+6+2]

Detailed Solution

(a) Issues with standard operators for TSP representation:

Crossover: Standard 1-point crossover slices two parents and swaps their tails. In TSP (where every city must be visited exactly once), swapping tails will cause some cities to be deleted completely, and others to be visited twice! This creates invalid, broken routes.

Mutation: Standard mutation randomly flips a bit (changes a city). Changing City 4 to City 9 means City 9 is visited twice and City 4 is erased. Again, invalidating the tour.

(b) Apply Order Crossover (OX1):

$P_1 = (1, 9, 2 \mid \mathbf{4, 6, 5, 7} \mid 8, 3)$

$P_2 = (4, 5, 9 \mid \mathbf{1, 8, 7, 6} \mid 2, 3)$

Cuts placed after index 3 and index 7.

Creating Child 1 (C_1): 1. Drop P_1 's middle slice directly down: $C_1 = (_, _, _ \mid 4, 6, 5, 7 \mid _, _)$

2. Look at P_2 , starting right after the second cut: 2, 3, 4, 5, 9, 1, 8, 7, 6. 3. Cross out the numbers already used in the middle slice (4, 6, 5, 7). You are left with: 2, 3, 9, 1, 8. 4. Fill these left-overs into C_1 starting right after the second cut, wrapping around:

$C_1 = (\mathbf{9, 1, 8,} \mid \mathbf{4, 6, 5, 7,} \mid \mathbf{2, 3})$

Creating Child 2 (C_2): 1. Drop P_2 's middle slice: $C_2 = (_, _, _ \mid 1, 8, 7, 6 \mid _, _)$ 2. Look at P_1 starting right after the cut: 8, 3, 1, 9, 2, 4, 6, 5, 7. 3. Cross out numbers already in the middle (1, 8, 7, 6). Leftovers: 3, 9, 2, 4, 5. 4. Fill them into C_2 :

$C_2 = (\mathbf{2, 4, 5,} \mid \mathbf{1, 8, 7, 6,} \mid \mathbf{3, 9})$

(c) Tournament selection & Large sizes:

How it works: A random subset of k individuals (tournament size) is picked from the population. They "compete," and the one with the best fitness wins and becomes a parent for the next generation.

If tournament size is too large: The selection pressure becomes too intense. The few elite individuals will win every single tournament. The entire next generation will just be clones of these elites, destroying genetic diversity. The algorithm will suffer from premature convergence (getting stuck in a local maximum).

□ Concept Breakdown (Never Forget This!)

Order Crossover (OX1) Concept:

Why do we do this weird wrapping-around math?

In a TSP problem, the relative order of cities matters more than their exact position. OX1 passes down a "chunk" of pure DNA from Parent 1, and then forces the remaining cities to line up in the exact sequence Parent 2 preferred, guaranteeing a valid child that honors both parents' strategies!